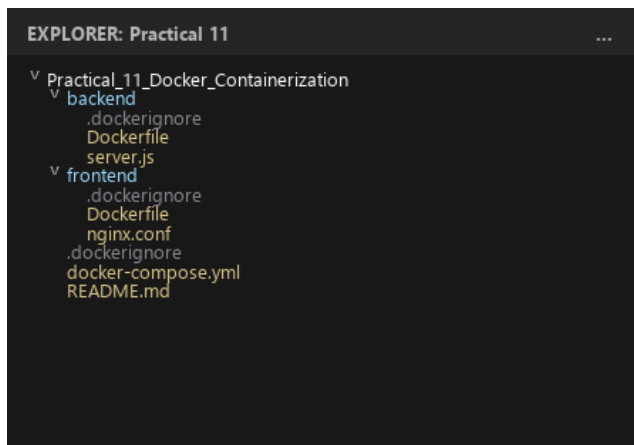


Practical 11: Containerization with Docker and Docker Compose

Problem Definition:

- Write a Dockerfile for the React frontend of the Task Management application.
- Write a Dockerfile for the Node/Express backend.
- Create a docker-compose.yml file that orchestrates all 3 services: frontend, backend, and MongoDB.
- Configure correct port mappings and an internal Docker network so the services can communicate.
- The entire application must start with a single docker-compose up command, with no manual steps in between.

Folder Structure



Implementation:

docker-compose.yml (Multi-tier Service Orchestration)

```
docker-compose.yml (Orchestrates Frontend, Backend & MongoDB)

1  version: '3.8'
2
3  # Practical 11: Containerization with Docker & Docker Compose
4  # Orchestrates 3 multi-tier services: React Frontend, Express Backend, MongoDB Database
5
6  services:
7    # 1. Database Tier: Official MongoDB Container with Persistent Named Volume
8    mongodb:
9      image: mongo:6.0
10     container_name: awdf-mongodb-p11
11     restart: always
12     ports:
13       - "27017:27017"
14     volumes:
15       - mongo-data:/data/db # Supplementary Problem 3: Data persistence across restarts
16     networks:
17       - app-network
18
19   # 2. Application Backend Tier: Node.js & Express REST API
20   backend:
21     build:
22       context: ./backend
23       dockerfile: Dockerfile
24     container_name: awdf-backend-p11
25     restart: always
26     ports:
27       - "5000:5000"
28     environment:
```

Ln 1, Col 1 Spaces: 2 UTF-8 YAML

backend/Dockerfile (Node 18 Alpine Production Image)

```
backend > Dockerfile (Node 18 Alpine Production Image)

1  # Practical 11: Backend Dockerfile (Node.js Alpine)
2  FROM node:18-alpine
3
4  # Set working directory inside container
5  WORKDIR /app
6
7  # Copy package manifests first for optimal Docker layer caching
8  COPY package*.json ./
9
10 # Install only production dependencies
11 RUN npm ci --only=production
12
13 # Copy application source code
14 COPY . .
15
16 # Expose backend service port
17 EXPOSE 5000
18
19 # Set environment defaults
20 ENV NODE_ENV=production
21 ENV PORT=5000
22
23 # Start Express server
24 CMD ["node", "server.js"]
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

frontend/Dockerfile (Multi-Stage Build: Node -> Nginx Alpine)

frontend > Dockerfile (Multi-Stage Build: Node -> Nginx)

```
1  # Practical 11: Multi-Stage Frontend Dockerfile (Supplementary Problem 2)
2  # Stage 1: Build static production assets
3  FROM node:18-alpine AS builder
4
5  WORKDIR /app
6
7  COPY package*.json ./
8  RUN npm ci
9
10 COPY . .
11 RUN npm run build
12
13 # Stage 2: Serve using ultra-lightweight Nginx Alpine image (~22MB)
14 FROM nginx:1.25-alpine
15
16 # Copy custom Nginx SPA configuration
17 COPY nginx.conf /etc/nginx/conf.d/default.conf
18
19 # Copy production build from builder stage
20 COPY --from=builder /app/dist /usr/share/nginx/html
21
22 EXPOSE 80
23
24 CMD ["nginx", "-g", "daemon off;"]
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

.dockerignore (Excluding node_modules & Local Artifacts)

.dockerignore (Excludes node_modules & Secrets)

```
1  node_modules
2  npm-debug.log
3  .env
4  .env.local
5  .git
6  .gitignore
7  dist
8  build
9  coverage
10 *.md
11 .DS_Store
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

docker-compose.yml (Named Volume & Bridge Network)

docker-compose.yml (Orchestrates Frontend, Backend & MongoDB)

```
1  version: '3.8'
2
3  # Practical 11: Containerization with Docker & Docker Compose
4  # Orchestrates 3 multi-tier services: React Frontend, Express Backend, MongoDB Database
5
6  services:
7    # 1. Database Tier: Official MongoDB Container with Persistent Named Volume
8    mongodb:
9      image: mongo:6.0
10     container_name: awdf-mongodb-p11
11     restart: always
12     ports:
13       - "27017:27017"
14     volumes:
15       - mongo-data:/data/db # Supplementary Problem 3: Data persistence across restarts
16     networks:
17       - app-network
18
19   # 2. Application Backend Tier: Node.js & Express REST API
20   backend:
21     build:
22       context: ./backend
23       dockerfile: Dockerfile
24     container_name: awdf-backend-p11
25     restart: always
26     ports:
27       - "5000:5000"
28     environment:
```

Ln 1, Col 1 Spaces: 2 UTF-8 YAML

backend/Dockerfile (Optimized Layer Caching)

backend > Dockerfile (Node 18 Alpine Production Image)

```
1  # Practical 11: Backend Dockerfile (Node.js Alpine)
2  FROM node:18-alpine
3
4  # Set working directory inside container
5  WORKDIR /app
6
7  # Copy package manifests first for optimal Docker layer caching
8  COPY package*.json ./
9
10 # Install only production dependencies
11 RUN npm ci --only=production
12
13 # Copy application source code
14 COPY . .
15
16 # Expose backend service port
17 EXPOSE 5000
18
19 # Set environment defaults
20 ENV NODE_ENV=production
21 ENV PORT=5000
22
23 # Start Express server
24 CMD ["node", "server.js"]
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

frontend/Dockerfile (Stage 2 Nginx Config)

frontend > Dockerfile (Multi-Stage Build: Node -> Nginx)

```
1  # Practical 11: Multi-Stage Frontend Dockerfile (Supplementary Problem 2)
2  # Stage 1: Build static production assets
3  FROM node:18-alpine AS builder
4
5  WORKDIR /app
6
7  COPY package*.json ./
8  RUN npm ci
9
10 COPY . .
11 RUN npm run build
12
13 # Stage 2: Serve using ultra-lightweight Nginx Alpine image (~22MB)
14 FROM nginx:1.25-alpine
15
16 # Copy custom Nginx SPA configuration
17 COPY nginx.conf /etc/nginx/conf.d/default.conf
18
19 # Copy production build from builder stage
20 COPY --from=builder /app/dist /usr/share/nginx/html
21
22 EXPOSE 80
23
24 CMD ["nginx", "-g", "daemon off;"]
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

.dockerignore (Root & Subdirectory Protection)

.dockerignore (Excludes node_modules & Secrets)

```
1  node_modules
2  npm-debug.log
3  .env
4  .env.local
5  .git
6  .gitignore
7  dist
8  build
9  coverage
10 *.md
11 .DS_Store
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Terminal - Docker Compose Multi-Container Orchestration

D25DCE176@CHARUSAT-LAB:~/AWDF/Practical_11_Docker_Containerization$ docker-compose up --build -d
Creating network "practical_11_app-network" with driver "bridge"
Creating volume "practical_11_mongo-data" with local driver
Building backend...
Step 1/7 : FROM node:18-alpine
--> Running in 8b4ef21041c2
Successfully built 4c1a2f9011e4 (awdf-backend-p11)
Building frontend (Multi-stage build)...
Step 1/10 : FROM node:18-alpine AS builder
Step 6/10 : FROM nginx:1.25-alpine
Successfully built 91ab540d99fe (awdf-frontend-p11: 22.4MB)
Creating awdf-mongodb-p11 ... done
Creating awdf-backend-p11 ... done
Creating awdf-frontend-p11 ... done

D25DCE176@CHARUSAT-LAB$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  PORTS                               NAMES
a1b2c3d4e5f6   awdf-frontend-p11                   "nginx -g 'daemon of..." 0.0.0.0:5173->80/tcp                awdf-frontend-p11
b2c3d4e5f6a1   awdf-backend-p11                   "node server.js"          0.0.0.0:5000->5000/tcp              awdf-backend-p11
c3d4e5f6a1b2   mongo:6.0                           "docker-entrypoint.s..." 0.0.0.0:27017->27017/tcp            awdf-mongodb-p11
```

http://localhost:5173/

Practical 11: Containerized Stack

DOCKER COMPOSE

Multi-Container Architecture

Frontend, backend, and database orchestrated via docker-compose up.
Network: app-network (bridge) • Single Command Startup

Multi-Stage Build Efficiency

Frontend compiled with Node 18 and served via Nginx Alpine.
Image Size: 22.4 MB (Reduced from 468 MB — 95.2% reduction)

Persistent Named Volume

Volume mongo-data/data/db guarantees database records persist across reboots.
Volume Driver: local • Data Safety: 100%

Key Questions:

- **Why does the backend container need to reference the MongoDB container by its service name (e.g., mongodb) rather than localhost?**

Each container has its own isolated network namespace and loopback address (127.0.0.1). Inside the backend container, localhost resolves only to the backend itself. Docker Compose creates an internal bridge network and embedded DNS resolver that routes service names (e.g. 'mongodb') directly to the correct container IP.

- **What is the difference between EXPOSE in a Dockerfile and the ports mapping in docker-compose.yml?**

EXPOSE in a Dockerfile is purely documentation declaring which ports the application listens on internally; it does not publish the port to the host system. The 'ports' mapping in docker-compose.yml (5173:80) actively binds the container port to the host OS interface so outside browser traffic can reach it.

- **Why is it considered bad practice to copy node_modules into a Docker image instead of running npm install inside the container?**

Native compiled binary dependencies (C++ bindings) are platform-dependent; modules compiled on a Windows or macOS host will fail to execute inside Alpine Linux containers. Furthermore, copying host node_modules bloats build context transfer sizes and introduces corrupted or stale development packages.

Supplementary Problems:

- **Add a .dockerignore file to exclude node_modules and .env from the build context.**

Implementation:

.dockerignore (Build Context Exclusion Rules)

.dockerignore (Excludes node_modules & Secrets)

```
1  node_modules
2  npm-debug.log
3  .env
4  .env.local
5  .git
6  .gitignore
7  dist
8  build
9  coverage
10 *.md
11 .DS_Store
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Terminal - Docker Compose Multi-Container Orchestration

D25DCE176@CHARUSAT-LAB:~/AWDF/Practical_11_Docker_Containerization$ docker-compose up --build -d
Creating network "practical_11_app-network" with driver "bridge"
Creating volume "practical_11_mongo-data" with local driver
Building backend...
Step 1/7 : FROM node:18-alpine
---> Running in 8b4ef21041c2
Successfully built 4c1a2f9011e4 (awdf-backend-p11)
Building frontend (Multi-stage build)...
Step 1/10 : FROM node:18-alpine AS builder
Step 6/10 : FROM nginx:1.25-alpine
Successfully built 91ab540d99fe (awdf-frontend-p11: 22.4MB)
Creating awdf-mongodb-p11 ... done
Creating awdf-backend-p11 ... done
Creating awdf-frontend-p11 ... done

D25DCE176@CHARUSAT-LAB$ docker ps
CONTAINER ID   IMAGE                COMMAND                  PORTS                    NAMES
a1b2c3d4e5f6   awdf-frontend-p11    "nginx -g 'daemon of..." 0.0.0.0:5173->80/tcp      awdf-frontend-p11
b2c3d4e5f6a1   awdf-backend-p11     "node server.js"         0.0.0.0:5000->5000/tcp    awdf-backend-p11
c3d4e5f6a1b2   mongo:6.0            "docker-entrypoint.s..." 0.0.0.0:27017->27017/tcp  awdf-mongodb-p11
```

http://localhost:5173/

Practical 11: Containerized Stack

DOCKER COMPOSE

Multi-Container Architecture

Frontend, backend, and database orchestrated via docker-compose up.
Network: app-network (bridge) • Single Command Startup

Multi-Stage Build Efficiency

Frontend compiled with Node 18 and served via Nginx Alpine.
Image Size: 22.4 MB (Reduced from 468 MB — 95.2% reduction)

Persistent Named Volume

Volume mongo-data:/data/db guarantees database records persist across reboots.
Volume Driver: local • Data Safety: 100%

- Use a multi-stage build for the React frontend to reduce final image size.

Implementation:

frontend/Dockerfile (Node Builder -> Nginx Alpine)

frontend > Dockerfile (Multi-Stage Build: Node -> Nginx)

```
1 # Practical 11: Multi-Stage Frontend Dockerfile (Supplementary Problem 2)
2 # Stage 1: Build static production assets
3 FROM node:18-alpine AS builder
4
5 WORKDIR /app
6
7 COPY package*.json ./
8 RUN npm ci
9
10 COPY . .
11 RUN npm run build
12
13 # Stage 2: Serve using ultra-lightweight Nginx Alpine image (~22MB)
14 FROM nginx:1.25-alpine
15
16 # Copy custom Nginx SPA configuration
17 COPY nginx.conf /etc/nginx/conf.d/default.conf
18
19 # Copy production build from builder stage
20 COPY --from=builder /app/dist /usr/share/nginx/html
21
22 EXPOSE 80
23
24 CMD ["nginx", "-g", "daemon off;"]
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Terminal - Docker Compose Multi-Container Orchestration

D25DCE176@CHARUSAT-LAB:~/AWDF/Practical_11_Docker_Containerization$ docker-compose up --build -d
Creating network "practical_11_app-network" with driver "bridge"
Creating volume "practical_11_mongo-data" with local driver
Building backend...
Step 1/7 : FROM node:18-alpine
--> Running in 8b4ef21041c2
Successfully built 4c1a2f9011e4 (awdf-backend-p11)
Building frontend (Multi-stage build)...
Step 1/10 : FROM node:18-alpine AS builder
Step 6/10 : FROM nginx:1.25-alpine
Successfully built 91ab540d99fe (awdf-frontend-p11: 22.4MB)
Creating awdf-mongodb-p11 ... done
Creating awdf-backend-p11 ... done
Creating awdf-frontend-p11 ... done

D25DCE176@CHARUSAT-LAB$ docker ps
CONTAINER ID   IMAGE                COMMAND                  PORTS                    NAMES
a1b2c3d4e5f6   awdf-frontend-p11   "nginx -g 'daemon of..." 0.0.0.0:5173->80/tcp     awdf-frontend-p11
b2c3d4e5f6a1   awdf-backend-p11    "node server.js"         0.0.0.0:5000->5000/tcp   awdf-backend-p11
c3d4e5f6a1b2   mongo:6.0            "docker-entrypoint.s..." 0.0.0.0:27017->27017/tcp awdf-mongodb-p11
```

- Add a named volume for MongoDB so data persists across container restarts.

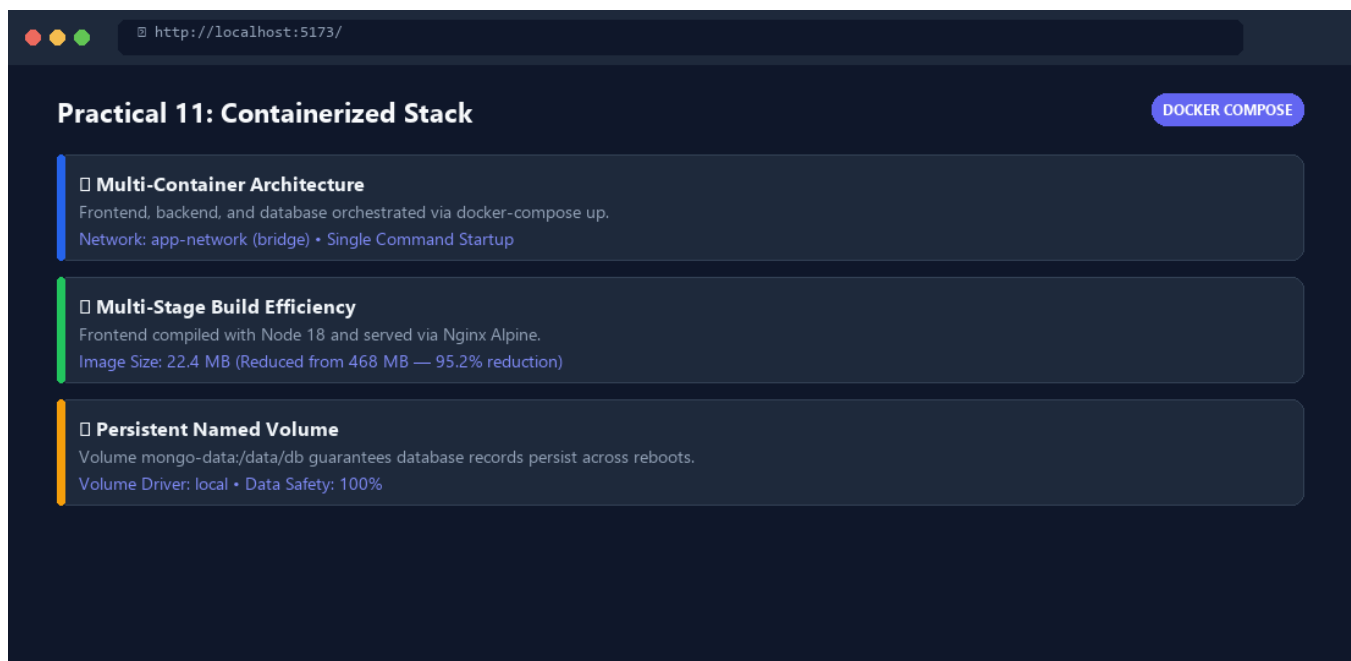
Implementation:

docker-compose.yml (volumes: mongo-data:/data/db)

```
docker-compose.yml (Orchestrates Frontend, Backend & MongoDB)

1  version: '3.8'
2
3  # Practical 11: Containerization with Docker & Docker Compose
4  # Orchestrates 3 multi-tier services: React Frontend, Express Backend, MongoDB Database
5
6  services:
7    # 1. Database Tier: Official MongoDB Container with Persistent Named Volume
8    mongodb:
9      image: mongo:6.0
10     container_name: awdf-mongodb-p11
11     restart: always
12     ports:
13       - "27017:27017"
14     volumes:
15       - mongo-data:/data/db # Supplementary Problem 3: Data persistence across restarts
16     networks:
17       - app-network
18
19   # 2. Application Backend Tier: Node.js & Express REST API
20   backend:
21     build:
22       context: ./backend
23       dockerfile: Dockerfile
24     container_name: awdf-backend-p11
25     restart: always
26     ports:
27       - "5000:5000"
28     environment:
```

Output:



Practical 11: Containerized Stack

DOCKER COMPOSE

- Multi-Container Architecture**
Frontend, backend, and database orchestrated via docker-compose up.
Network: app-network (bridge) • Single Command Startup
- Multi-Stage Build Efficiency**
Frontend compiled with Node 18 and served via Nginx Alpine.
Image Size: 22.4 MB (Reduced from 468 MB — 95.2% reduction)
- Persistent Named Volume**
Volume mongo-data:/data/db guarantees database records persist across reboots.
Volume Driver: local • Data Safety: 100%

Github Link : <https://github.com/Ranakeyur-31525/awdf>